TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA TOÁN - TIN HỌC



BÁO CÁO THỰC HÀNH

Môn học: Xử lý dữ liệu lớn

Bài thực hành L04

Nạp dữ liệu từ cơ sở dữ liệu OLTP vào HDFS: Sqoop và Spark JDBC, nạp toàn phần và nạp gia tăng

Sinh viên thực hiện: Nguyễn Thị Huyền Trang
Mã số sinh viên: 23280092
Lớp: 23KDL

TP. Hồ Chí Minh, Tháng 10 năm 2026

Mục lục

Tóm tắt **3**

1 Giới thiệu **3**

2 Cơ sở lý thuyết **4**
2.1 Nạp toàn phần và nạp gia tăng 4
2.2 Cách Sqoop chia split 4
2.3 Đọc JDBC song song trong Spark 4
2.4 Watermark, change log và snapshot 4
2.5 Change Data Capture 5

3 Môi trường và phương pháp thực nghiệm **5**
3.1 Môi trường 5
3.2 Dữ liệu 6
3.3 Quy trình 7

4 Kết quả thực nghiệm **7**
4.1 Sqoop nạp toàn phần bảng `orders` 7
4.2 Sqoop với cột chia bị lệch 9
4.3 Sqoop nạp gia tăng, merge và export 10
4.4 Spark JDBC: đọc song song và ghi Parquet 13
4.5 So sánh Sqoop và Spark JDBC 17
4.6 Spark nạp gia tăng bằng watermark 18

5 Thảo luận **21**
5.1 Sqoop chạy câu SQL nào trước khi import, và từ đó ra 4 split bằng cách nào? . 21
5.2 Vì sao các file `customers` không đều, và nếu dùng `--split-by customer_id` thì sao? 21
5.3 Với `lowerBound=1, upperBound=5003, numPartitions=4`, Spark có đọc dòng `order_seq` = 9,999 không? 21
5.4 Vì sao phải chốt cận trên trước khi đọc, và chỉ lưu watermark sau khi ghi xong? 21
5.5 Chế độ `append` hay `lastmodified` bắt được UPDATE, và chế độ nào bắt được DELETE? 22
5.6 Watermark trên `updated_at` không giải quyết được gì, và CDC khác ở đâu? . . 22
5.7 Điều gì xảy ra với cơ sở dữ liệu nguồn khi đặt `-m 50` hoặc `numPartitions=50`? 22
5.8 Hạn chế của thực nghiệm 23

6 Kết luận **23**

Tài liệu tham khảo **23**

Phụ lục A: Danh mục tệp nộp kèm **24**

Phụ lục B: Trích đoạn mã nguồn **24**

Danh mục hình

1 Trạng thái các container của stack L04 (`docker compose ps`) và danh sách 6 bảng nguồn trong PostgreSQL; container `jdbc-driver` kết thúc với mã 0 sau khi tải driver. 6
2 Kết quả Sqoop nạp toàn phần `orders`: bounding query, 4 split, 1.009 MB trong 25.2306 s và 1,250 dòng mỗi file. 8
3 Nhật ký PostgreSQL ghi lại bounding query và 4 câu SELECT theo khoảng `order_seq` do các map task của Sqoop gửi. 9
4 JobHistory của job `job_1793356967101_0001` (`orders.jar`): 4 map task thành công, không có reduce task. 9
5 Số dòng mỗi file khi nạp `customers` chia theo `customer_zip_code_prefix`, thể hiện dữ liệu bị lệch. 10
6 Sqoop nạp `append` lần đầu, nạp `lastmodified` cơ sở và tạo saved job `orders_incr`. 12
7 Sau `simulate_changes` lần 1: saved job lấy 3 dòng mới, `lastmodified` kèm merge lấy 5 dòng, và export 5,000 record về PostgreSQL. 12
8 YARN ResourceManager UI: các MapReduce job do Sqoop nộp, tất cả ở trạng thái FINISHED/SUCCEEDED. 13
9 JobHistory Server liệt kê các job Sqoop đã hoàn tất. 13
10 Kết quả `spark-submit --step full`: số partition, thời gian đếm, số dòng mỗi partition, kết quả ghi Parquet và physical plan có `PartitionFilters`. 15
11 Nhật ký PostgreSQL ghi lại các câu `SELECT 1 FROM orders WHERE ...` theo khoảng do Spark sinh ra. 15
12 Spark UI, trang Jobs của ứng dụng `L04-jdbc-full`. 16
13 Spark UI, trang Stages: các stage đọc JDBC song song và stage ghi Parquet đều có 4 task. 17
14 Đầu ra Parquet phân vùng theo tháng trên HDFS và so sánh dung lượng với đầu ra văn bản của Sqoop. 17
15 Ba lần chạy Spark `--step incremental`: lần 1 đọc 5,003 dòng, lần 2 đọc 5 dòng thay đổi, lần 3 đọc 0 dòng. 20

Danh mục bảng

1 Môi trường thực nghiệm (stack `l04_ingestion`). 5
2 Phân bố trạng thái của 5,000 đơn hàng ban đầu. 6

3 Các split của Sqoop khi nạp orders với --split-by order_seq -m 4. . . . 8
4 Số dòng mỗi file khi chia theo cột phân bố đều và cột bị lệch. 10
5 Phân bố customer_zip_code_prefix theo width_bucket(..., 1003, 99990, 4). 10
6 Kết quả các bước nạp gia tăng, merge và export của Sqoop. 11
7 So sánh các cách đọc bảng orders bằng Spark JDBC. 14
8 Số dòng các bảng phụ được Spark nạp sang Parquet. 14
9 So sánh Sqoop và Spark JDBC khi nạp bảng orders. 18
10 Kết quả ba lần chạy Spark nạp gia tăng (thời điểm ngày 2026-10-30). 19
11 Danh mục tệp nộp kèm báo cáo L04. 24

Tóm tắt

Bài thực hành này nạp dữ liệu theo lô từ PostgreSQL (bộ dữ liệu Olist, 6 bảng, 5,000 đơn hàng) vào HDFS bằng hai công cụ: Apache Sqoop 1.4.7 chạy MapReduce trên YARN và Spark 4.1.3 qua nguồn dữ liệu JDBC. Chúng tôi bật log_statement=all trên PostgreSQL để đọc trực tiếp câu SQL mà mỗi công cụ gửi xuống. Khi chia theo order_seq, Sqoop tạo 4 khoảng đều nhau (1,250 dòng mỗi map); chia theo customer_zip_code_prefix thì lệch nặng (2,450 so với 427 dòng). Spark JDBC đọc song song 4 partition, thời gian count() giảm từ 1.17 s xuống 0.22 s, và đầu ra Parquet phân vùng theo tháng chiếm 680.4 KB so với 1.0 MB văn bản của Sqoop. Ở chế độ gia tăng, append chỉ bắt được 3 dòng INSERT; lastmodified và watermark trên updated_at bắt được 5 dòng (3 INSERT, 2 UPDATE), lần chạy không có thay đổi đọc 0 dòng. Spark JDBC thay được Sqoop, nhưng mọi cách nạp dựa trên truy vấn đều bỏ sót DELETE và các phiên bản trung gian, nên vẫn cần CDC.

Từ khóa: Sqoop, Spark JDBC, PostgreSQL, HDFS, Parquet, split-by, partitionColumn, incremental load, watermark, CDC.

1 Giới thiệu

Dữ liệu nghiệp vụ phần lớn nằm trong các cơ sở dữ liệu giao dịch (OLTP). Muốn phân tích trên nền tảng dữ liệu lớn thì phải định kỳ chép dữ liệu đó sang HDFS. Có hai yêu cầu đi kèm: đọc song song đủ nhanh mà không làm sập cơ sở dữ liệu nguồn, và ở các lần sau chỉ đọc phần đã thay đổi.

Công cụ truyền thống trong hệ Hadoop là Apache Sqoop [1]. Dự án này đã vào Apache Attic từ tháng 6 năm 2021, nên hiện nay người ta thường dùng Spark JDBC [2] hoặc CDC thay thế. Bài này nhắm tới bốn việc:

1. Chạy Sqoop nạp toàn phần với --split-by và -m, đọc câu SQL Sqoop sinh ra để hiểu cách chia split, và so sánh cột phân bố đều (order_seq) với cột lệch (customer_zip_

`code_prefix`).

2. Chạy Sqoop nạp gia tăng ở chế độ `append` và `lastmodified`, kèm saved job, `--merge-key` và `sqoop export`.
3. Viết phần còn thiếu của `spark_jdbc_ingest.py`: đọc JDBC song song, ghi Parquet phân vùng theo tháng, nạp gia tăng bằng watermark và dựng snapshot khử trùng lặp.
4. So sánh hai công cụ và chỉ ra giới hạn của nạp gia tăng dựa trên truy vấn so với CDC.

2 Cơ sở lý thuyết

2.1 Nạp toàn phần và nạp gia tăng

Nạp toàn phần đọc lại cả bảng mỗi lần, đơn giản và luôn nhất quán nhưng tốn kém khi bảng lớn. Nạp gia tăng chỉ đọc dòng mới hoặc dòng đã đổi, dựa vào một cột đơn điệu: khóa tăng dần (bắt INSERT) hoặc cột thời điểm cập nhật (bắt cả INSERT và UPDATE). Giá trị lớn nhất đã xử lý được lưu lại làm watermark cho lần chạy sau [5].

2.2 Cách Sqoop chia split

Mỗi lệnh `import` được biên dịch thành một MapReduce job chỉ có pha map [1], [6]. Sqoop chạy một *bounding query* `SELECT MIN(col), MAX(col)` trên cột `--split-by`, rồi chia đoạn $[\min, \max]$ thành `-m` khoảng có độ rộng giá trị bằng nhau. Mỗi map mở một kết nối JDBC và chạy `SELECT ... WHERE col >= lo AND col < hi` (khoảng cuối dùng <=). Số dòng mỗi map chỉ đều khi giá trị của cột chia phân bố đều.

2.3 Đọc JDBC song song trong Spark

Nguồn JDBC của Spark SQL [2], [3] nhận `partitionColumn`, `lowerBound`, `upperBound` và `numPartitions`. Spark tính bước $s = (\text{upper} - \text{lower})/n$ và sinh n mệnh đề WHERE; partition đầu là `col < lower + s OR col IS NULL`, partition cuối là `col >= lower + (n-1)s`, cả hai không có cận ngoài. Như vậy hai cận chỉ quyết định bước chia chứ không lọc dữ liệu. Cột chia phải có kiểu số, ngày hoặc timestamp; mỗi partition là một task của RDD [4] và một kết nối riêng.

2.4 Watermark, change log và snapshot

Pipeline gia tăng thường ghi các dòng thay đổi vào một change log chỉ nối thêm, rồi dựng snapshot bằng cách giữ phiên bản mới nhất của mỗi khóa, ví dụ với `row_number()`. Cận trên của cửa sổ phải chốt trước khi đọc, và watermark chỉ lưu sau khi ghi xong. Kết quả là ngữ nghĩa *at-least-once* cộng với bước khử trùng lặp idempotent [5].

2.5 Change Data Capture

CDC dựa trên nhật ký giao dịch không truy vấn bảng mà đọc write-ahead log (WAL) qua *logical decoding* [7]. Các công cụ như Debezium [8] phát ra mọi INSERT, UPDATE và DELETE theo thứ tự commit, kèm ảnh trước và sau của dòng.

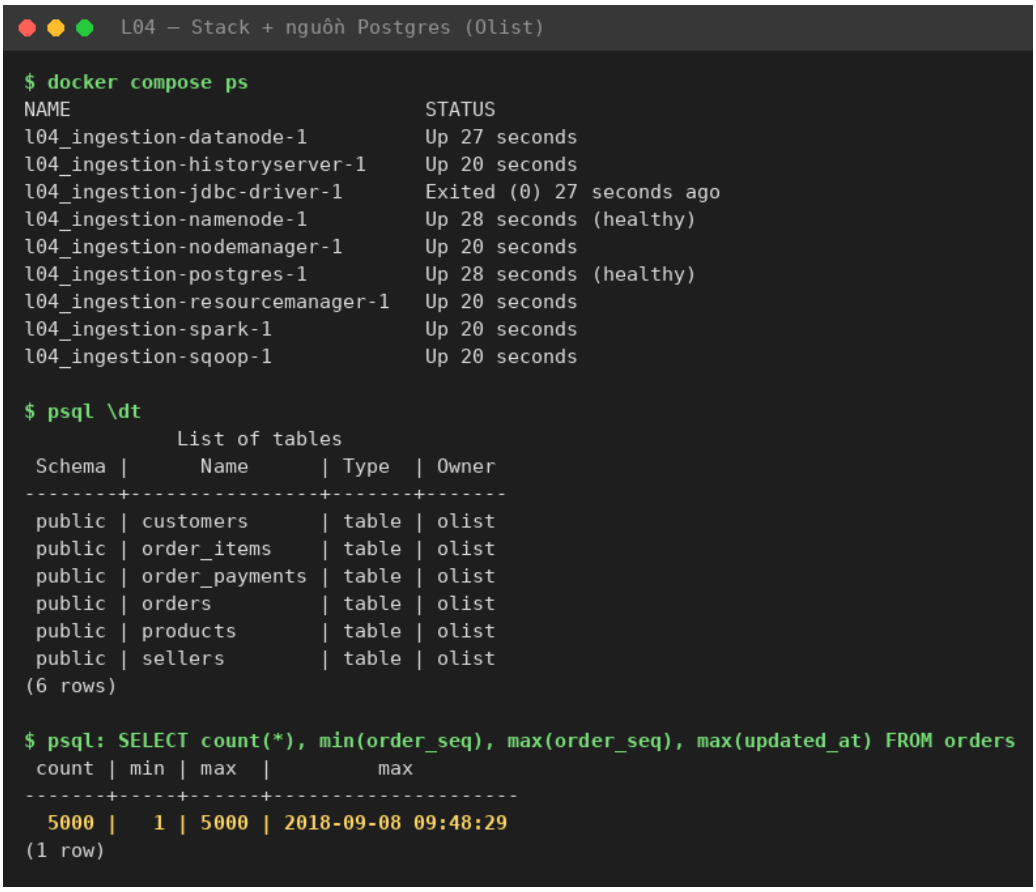
3 Môi trường và phương pháp thực nghiệm

3.1 Môi trường

Toàn bộ hệ thống chạy bằng Docker Compose trên một máy (Bảng 1). Lúc khởi động, container `jdbc-driver` chỉ tải driver rồi thoát với mã 0; ban đầu chúng tôi tưởng nó lỗi, nhưng đó là hành vi đúng (Hình 1).

Bảng 1. Môi trường thực nghiệm (stack `l04_ingestion`).

Thành phần	Phiên bản / cấu hình	Vai trò
PostgreSQL	18.6, `log_statement=all`	Cơ sở dữ liệu nguồn Olist, ghi lại mọi câu SQL
HDFS	3.4.3, 1 NameNode + 1 DataNode	Đích lưu trữ
YARN	ResourceManager, NodeManager, JobHistory Server	Chạy MapReduce job của Sqoop
Sqoop	1.4.7 (image khóa học `de-labs/sqoop:1.4.7`)	Công cụ nạp truyền thống
Spark	4.1.3, local mode, Spark UI cổng 4040	Công cụ nạp hiện đại
JDBC driver	PostgreSQL JDBC (`/jars/postgresql.jar`)	Kết nối cho Sqoop và Spark



Hình 1. Trạng thái các container của stack L04 (`docker compose ps`) và danh sách 6 bảng nguồn trong PostgreSQL; container `jdbc-driver` kết thúc với mã 0 sau khi tải driver.

3.2 Dữ liệu

Nguồn là cơ sở dữ liệu Olist tổng hợp gồm 6 bảng: `customers`, `orders`, `order_items`, `order_payments`, `products`, `sellers`. Bảng `orders` có 5,000 dòng, `order_seq` chạy từ 1 đến 5000, `max(updated_at)` là `2018-09-08 09:48:29`. `order_seq` là khóa thay thế kiểu số, tăng theo thời gian mua; các khóa gốc như `order_id`, `customer_id` là chuỗi hash 32 ký tự. Mỗi bảng có cột `updated_at` do trigger tự cập nhật khi UPDATE. Phân bố trạng thái đơn hàng lấy bằng `sqoop eval` nằm ở Bảng 2.

Bảng 2. Phân bố trạng thái của 5,000 đơn hàng ban đầu.

`order_status`	Số đơn
delivered	4,793
shipped	102
unavailable	41
canceled	30
processing	19
invoiced	11

order_status	Số đơn
approved	3
created	1

Script `sql/simulate_changes.sql` mô phỏng một ngày nghiệp vụ: chèn 3 khách hàng và 3 đơn mới (trạng thái `created`), đổi 2 đơn `shipped` đầu tiên thành `delivered`. Chúng tôi chạy script hai lần: lần 1 trước phần Sqoop gia tăng (tạo `order_seq` 5001–5003, cập nhật đơn 139 và 278), lần 2 giữa hai lần chạy Spark gia tăng (tạo 5004–5006, cập nhật đơn 337 và 355). Vì thế lúc chạy Spark, `orders` và `customers` đã có 5,003 dòng chứ không phải 5,000.

3.3 Quy trình

1. Khởi động stack, kiểm tra 6 bảng và thống kê `orders`.
2. Sqoop phần 1 (`sqoop_part1.sh`): `list-tables`, `eval`; nạp toàn phần `orders` với `--split-by order_seq -m 4`; nạp `customers` với `--split-by customer_zip_code_prefix -m 4`; nạp append lần đầu với `-m 2`, `--last-value 0`; nạp `lastmodified` cơ sở với `-m 1`, `--last-value '1970-01-01 00:00:00'`; tạo saved job `orders_incr`. Đếm số dòng từng file `part-m-*`.
3. Chạy `simulate_changes.sql` lần 1.
4. Sqoop phần 2: `sqoop job --exec orders_incr`; `lastmodified` với `--merge-key order_id`; `sqoop export` từ `orders_full` sang bảng mới `orders_copy` với `-m 2`.
5. Spark `--step full`: đọc naive, đọc song song theo `order_seq` và theo `order_purchase_timestamp`, ghi Parquet phân vùng `purchase_month`, kiểm tra partition pruning, nạp 5 bảng còn lại.
6. Spark `--step incremental` ba lần: lần 1; chạy `simulate_changes.sql` lần 2; lần 2; lần 3 khi không có thay đổi.
7. Trong suốt quá trình, thu nhật ký PostgreSQL để đối chiếu câu SQL; chụp YARN UI, JobHistory và Spark UI.

Các tham số thay đổi giữa các thí nghiệm là cột chia (`order_seq`, `customer_zip_code_prefix`, `order_purchase_timestamp`), mức song song (`-m 1/2/4`, `numPartitions` 1 hoặc 4) và chế độ gia tăng.

4 Kết quả thực nghiệm

4.1 Sqoop nạp toàn phần bảng `orders`

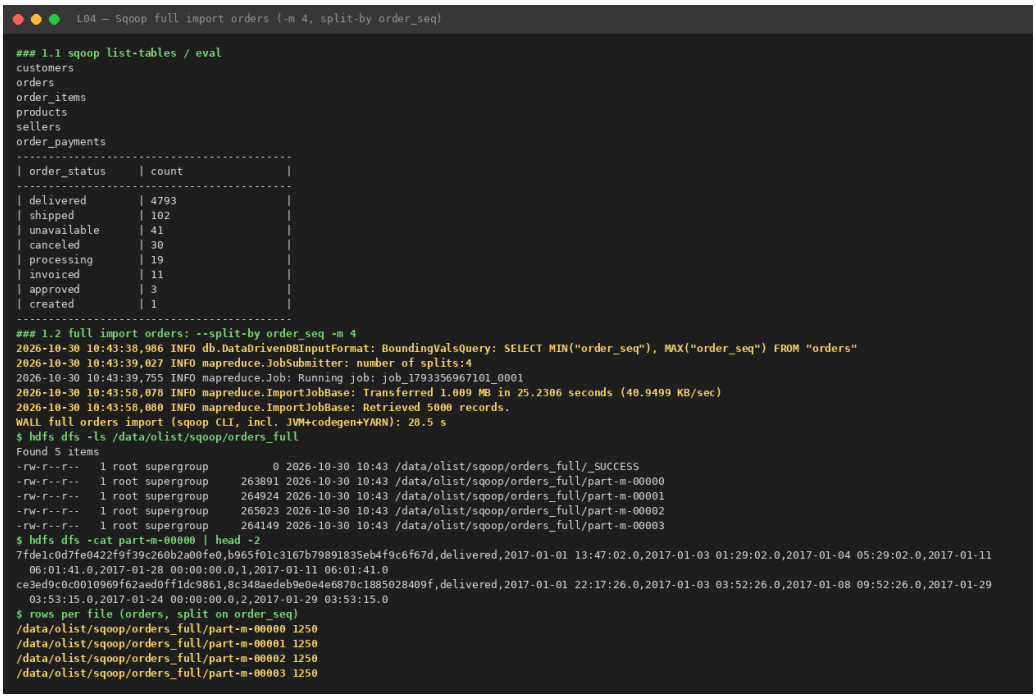
Sqoop gửi `SELECT MIN("order_seq"), MAX("order_seq") FROM "orders"`, nhận (1, 5000) và báo `number of splits:4`. Nhật ký PostgreSQL có đúng 4 câu SELECT của 4 map (Bảng 3, Hình 3). Job `job_1793356967101_0001` báo `Transferred 1.009 MB in 25.2306 seconds`

`(40.9499 KB/sec)` và `Retrieved 5000 records`. Cả lệnh CLI mất 28.5 s, vì còn phải khởi động JVM, sinh và biên dịch lớp record rồi nộp job lên YARN.

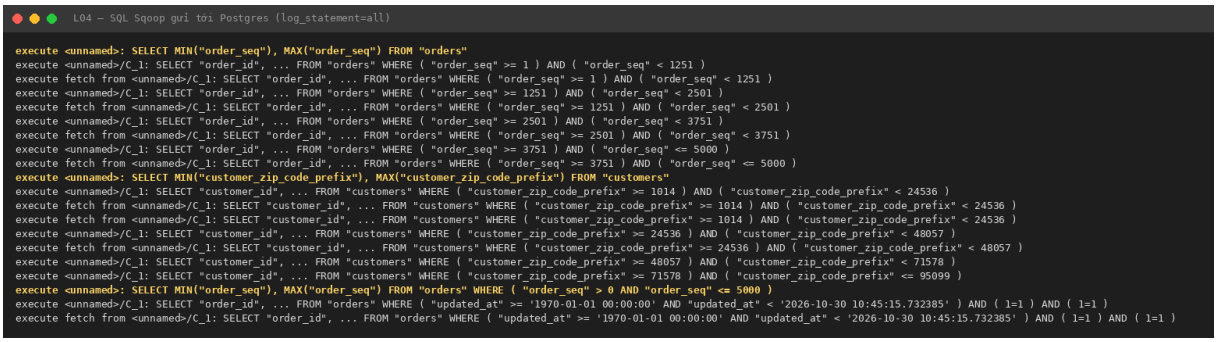
Bảng 3. Các split của Sqoop khi nạp `orders` với `--split-by order_seq -m 4`.

Map	Điều kiện WHERE	Số dòng	Kích thước file
0	`order_seq >= 1 AND order_seq < 1251`	1,250	263,891 B
1	`order_seq >= 1251 AND order_seq < 2501`	1,250	264,924 B
2	`order_seq >= 2501 AND order_seq < 3751`	1,250	265,023 B
3	`order_seq >= 3751 AND order_seq <= 5000`	1,250	264,149 B

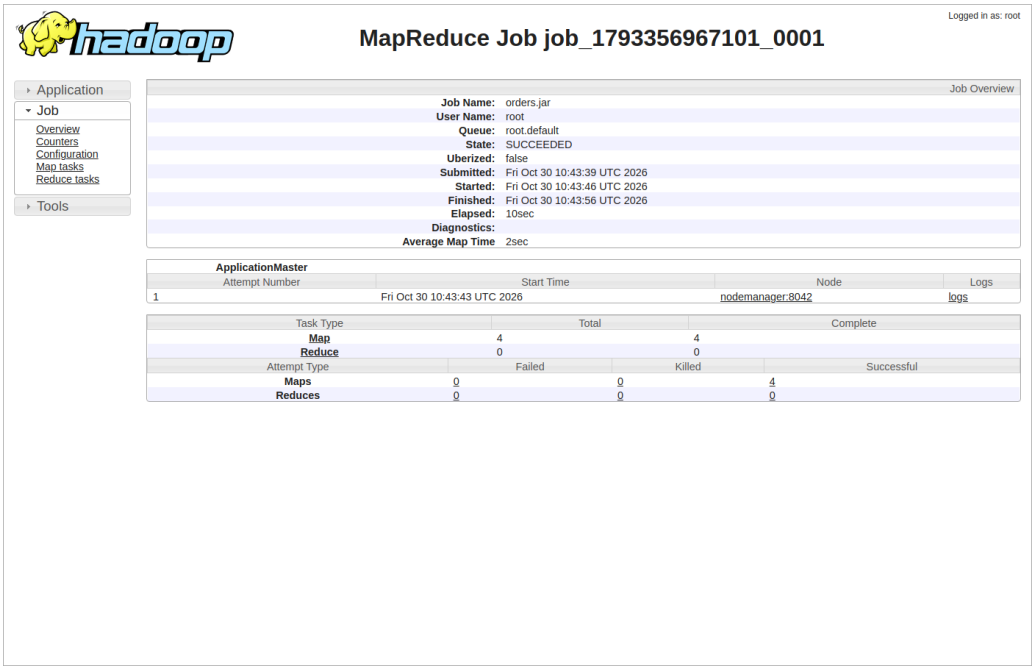
Bốn file `part-m-00000` đến `part-m-00003` gần như bằng nhau cả về số dòng lẫn kích thước (Hình 2). Dữ liệu là văn bản CSV, timestamp có dạng `2017-01-03 01:29:02.0`, NULL được ghi là `\N`. JobHistory xác nhận job có 4 map, 0 reduce, tất cả thành công (Hình 4).



Hình 2. Kết quả Sqoop nạp toàn phần `orders`: bounding query, 4 split, 1.009 MB trong 25.2306 s và 1,250 dòng mỗi file.



Hình 3. Nhật ký PostgreSQL ghi lại bounding query và 4 câu SELECT theo khoảng `order_seq` do các map task của Sqoop gửi.



Hình 4. JobHistory của job `job_1793356967101_0001` (`orders.jar`): 4 map task thành công, không có reduce task.

4.2 Sqoop với cột chia bị lệch

Với `customers`, chúng tôi chia theo `customer_zip_code_prefix -m 4`. Bounding query trả về (1014, 95099), và nhật ký PostgreSQL cho thấy 4 khoảng rộng xấp xỉ 23,521 với các điểm cắt 24536, 48057 và 71578. Số dòng mỗi file nằm ở Bảng 4, đặt cạnh trường hợp `orders` để dễ so.

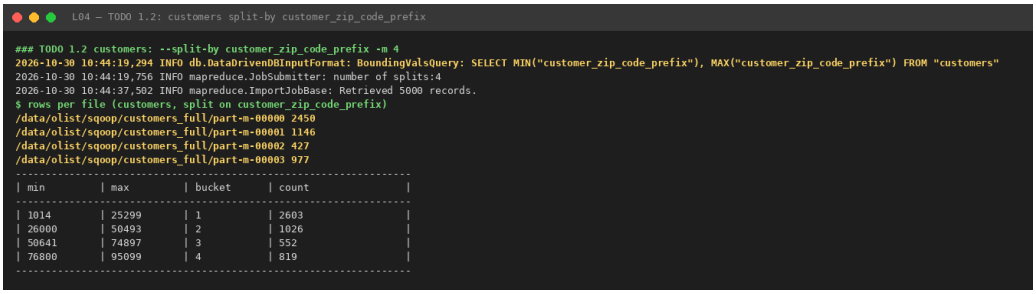
Bảng 4. Số dòng mỗi file khi chia theo cột phân bố đều và cột bị lệch.

Bảng / cột split	part-m-00000	part-m-00001	part-m-00002	part-m-00003
`orders` / `order_seq` (1–5000)	1,250	1,250	1,250	1,250
`customers` / `customer_zip_code_prefix` (1014–95099)	**2,450**	1,146	**427**	977

Split đầu nhận gần nửa bảng, gấp khoảng 5.7 lần split thứ ba (Hình 5). Để kiểm tra độc lập, chúng tôi đếm zip code bằng `width_bucket` trên 4 thùng (Bảng 5) và thấy cùng xu hướng: thùng chứa vùng São Paulo (01xxx–19xxx) chiếm phần lớn khách hàng.

Bảng 5. Phân bố `customer_zip_code_prefix` theo `width_bucket(..., 1003, 99990, 4)`.

Thùng	min zip	max zip	Số khách hàng
1	1014	25299	2,603
2	26000	50493	1,026
3	50641	74897	552
4	76800	95099	819



Hình 5. Số dòng mỗi file khi nạp `customers` chia theo `customer_zip_code_prefix`, thể hiện dữ liệu bị lệch.

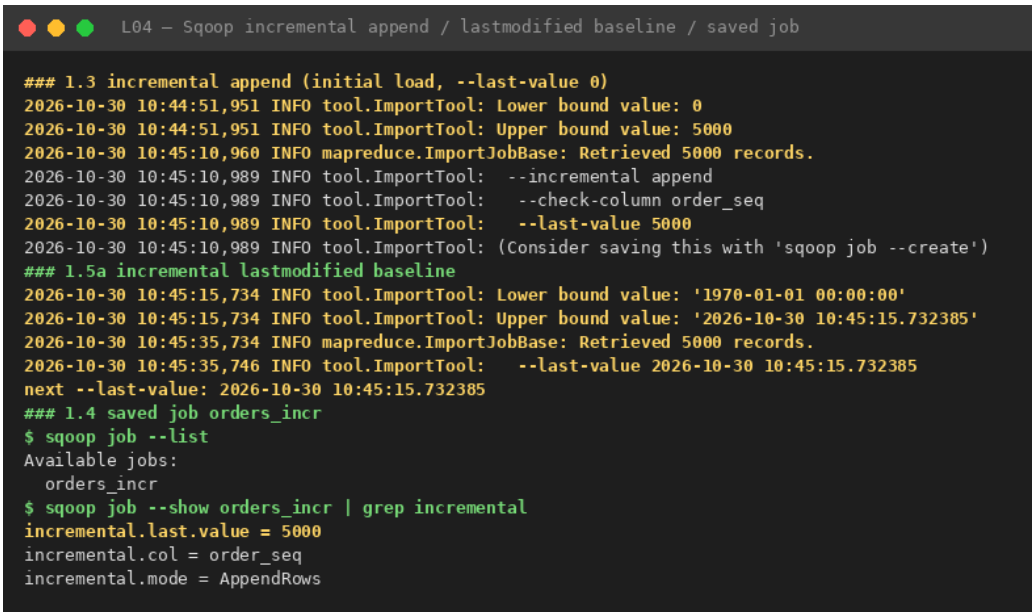
4.3 Sqoop nạp gia tăng, merge và export

Chuỗi thao tác gia tăng được tóm ở Bảng 6. Lần `append` đầu với `-m 2` có điều kiện `order_seq > 0 AND order_seq <= 5000` và hai split `[1, 2501)`, `[2501, 5000]`. Cuối lệnh, Sqoop gợi ý `--last-value 5000` và đề nghị lưu thành saved job. Saved job `orders_incr` lưu `incremental.col = order_seq` và `incremental.mode = AppendRows` trong metastore của Sqoop.

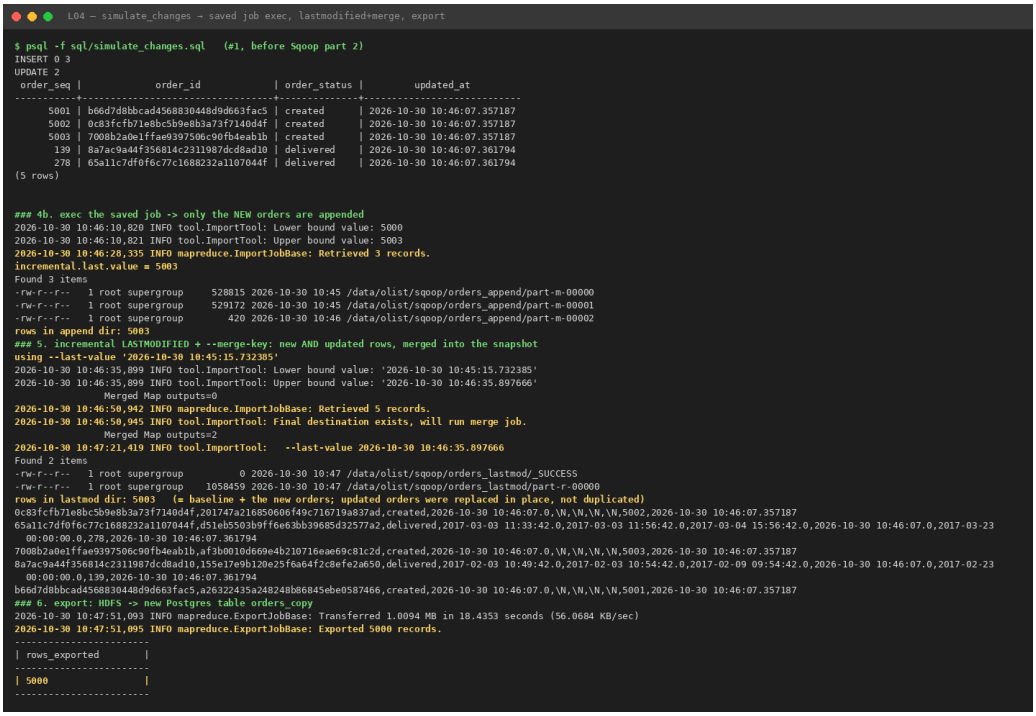
Bảng 6. Kết quả các bước nạp gia tăng, merge và export của Sqoop.

Bước	Cửa sổ	Kết quả
append, lần đầu (-m 2)	lower 0, upper 5000	5,000 dòng, gợi ý --last-value 5000
lastmodified, cơ sở (-m 1)	'1970-01-01 00:00:00' đến '2026-10-30 10:45:15.732385'	5,000 dòng
Saved job orders_incr		incremental.last.value = 5000
simulate_changes lần 1		tạo order_seq 5001–5003; đơn 139 và 278 chuyển delivered
sqoop job --exec orders_incr	lower 5000, upper 5003	**3** dòng; last.value thành 5003; thư mục có 5,003 dòng
lastmodified + --merge-key order_id	'...10:45:15.732385' đến '...10:46:35.897666'	**5** dòng; merge job tạo part-r-00000 5,003 dòng
sqoop export sang orders_copy (-m 2)		1.0094 MB trong 18.4353 s, 5,000 record

Saved job chạy ... WHERE ("order_seq" > 5000 AND "order_seq" <= 5003) và chỉ thêm một file nhỏ part-m-00002 (420 B) cạnh hai file 528,815 B và 529,172 B của lần đầu. Đơn 139 và 278 vừa cập nhật không được nạp lại, vì append chỉ nhìn khóa mới. lastmodified thì lấy đủ 5 dòng. Do thư mục đích đã tồn tại, Sqoop chạy thêm một merge job (Merged Map outputs=2) để thay bản cũ của cùng order_id bằng bản mới. Kết quả là một file part-r-00000 (1,058,459 B), không trùng lặp, trong đó đơn 139 và 278 mang trạng thái delivered và updated_at mới. Một chi tiết chúng tôi không để ý cho tới khi đọc log: lastmodified dùng cửa sổ nửa mở updated_at >= lower AND updated_at < upper, ngược với Spark ở phần sau. Các bước này ở Hình 6 và Hình 7.

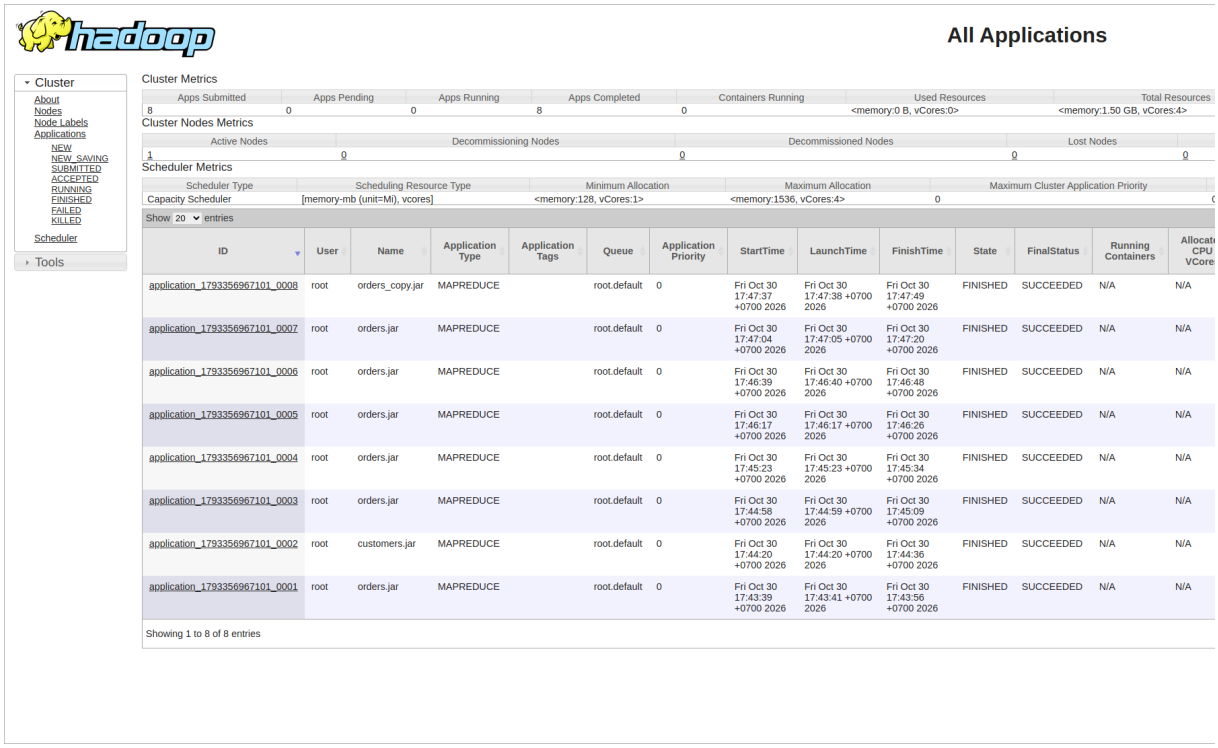


Hình 6. Sqoop nạp append lần đầu, nạp `lastmodified` cơ sở và tạo saved job `orders_incr`.

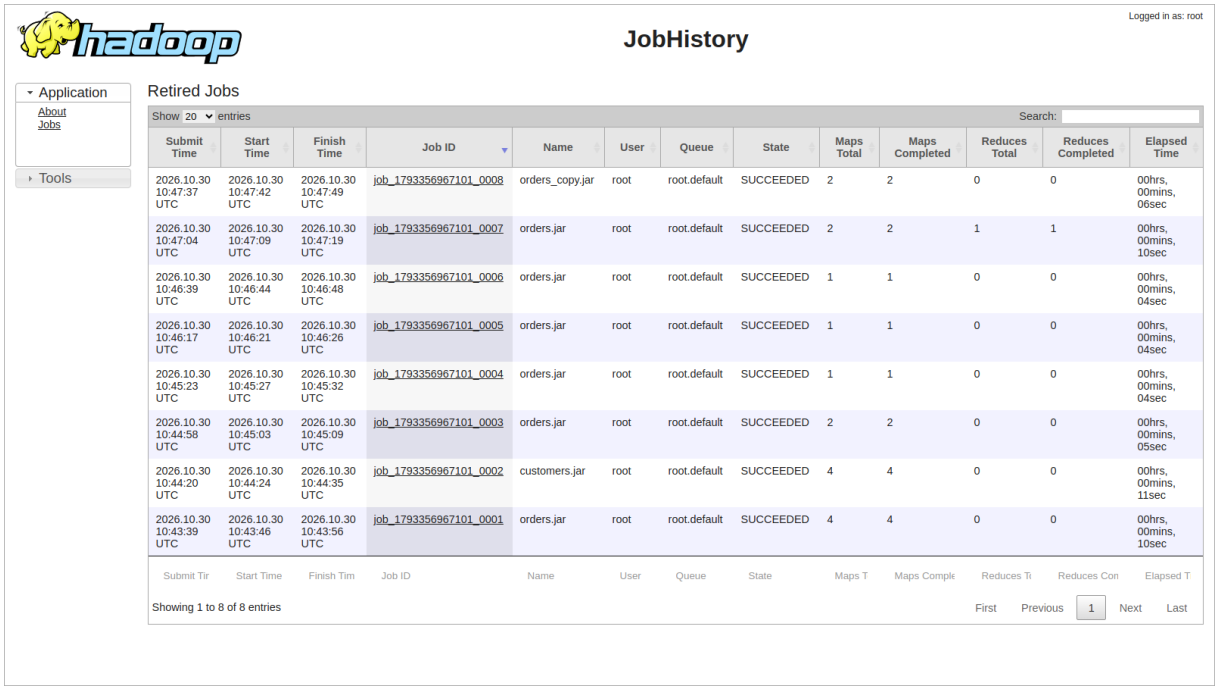


Hình 7. Sau `simulate_changes` lần 1: saved job lấy 3 dòng mới, `lastmodified` kèm merge lấy 5 dòng, và export 5,000 record về PostgreSQL.

Mỗi lệnh Sqoop là một ứng dụng MapReduce trên YARN. ResourceManager liệt kê 8 ứng dụng (`orders.jar`, `customers.jar`, `orders_copy.jar`) đều kết thúc thành công (Hình 8); JobHistory tương ứng ở Hình 9.



Hình 8. YARN ResourceManager UI: các MapReduce job do Sqoop nộp, tất cả ở trạng thái FINISHED/SUCCEEDED.



Hình 9. JobHistory Server liệt kê các job Sqoop đã hoàn tất.

4.4 Spark JDBC: đọc song song và ghi Parquet

Script lấy cận bằng `SELECT min(order_seq) AS lo, max(order_seq) AS hi FROM orders`, nhận `1 .. 5003`, rồi đọc theo ba cách (Bảng 7).

Bảng 7. So sánh các cách đọc bảng `orders` bằng Spark JDBC.

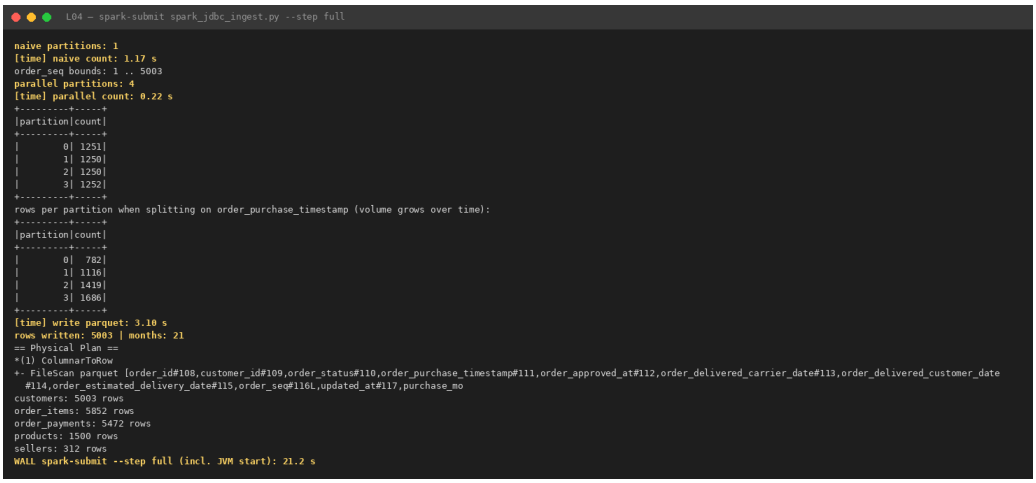
Cách đọc	Số partition	Thời gian `count()`	Số dòng mỗi partition
Naive (không có `partitionColumn`)	1	1.17 s	5,003 dòng trong 1 task
`partitionColumn=order_seq`, bounds 1..5003	4	0.22 s	1,251 / 1,250 / 1,250 / 1,252
`partitionColumn=order_purchase_timestamp`, 2017-01-01 đến 2018-09-01	4	không đo	782 / 1,116 / 1,419 / 1,686

Nhật ký PostgreSQL (Hình 11) có 4 câu do Spark sinh: `WHERE "order_seq" < 1252 or "order_seq" is null`, `>= 1252 AND < 2502`, `>= 2502 AND < 3752` và `WHERE "order_seq" >= 3752`. Bước chia là $(5003 - 1)/4 \approx 1250$, nên partition đầu có 1,251 dòng (1 đến 1251) và partition cuối có 1,252 dòng (3752 đến 5003). Vì lệnh là `count()`, Spark chỉ gửi `SELECT 1`, tức column pruning đã được đẩy xuống SQL. Với cột timestamp, partition đầu là `< '2017-06-02 00:00:00' or ... is null` và partition cuối là `>= '2018-04-02 00:00:00'`; số dòng tăng dần theo thời gian vì lượng đơn hàng của Olist tăng dần.

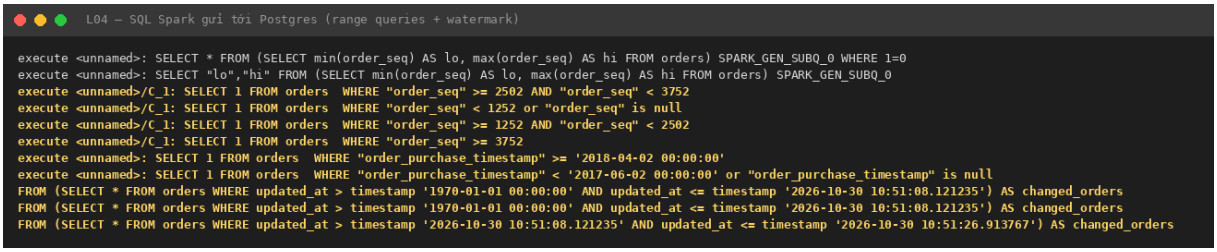
Ghi Parquet với `partitionBy("purchase_month")` mất 3.10 s, tạo 21 thư mục tháng (20 tháng lịch sử và tháng của các đơn mới) chứa đủ 5,003 dòng. Truy vấn lọc `purchase_month = '2018-08'` có physical plan chứa `PartitionFilters: [isnotnull(purchase_month#118), (purchase_month#118 = 2018-08)]`, nghĩa là partition pruning có tác dụng. Năm bảng còn lại được nạp mỗi bảng một partition (Bảng 8). Toàn bộ `spark-submit --step full`, tính cả khởi động JVM, mất 21.2 s (Hình 10).

Bảng 8. Số dòng các bảng phụ được Spark nạp sang Parquet.

Bảng	Số dòng
customers	5,003
order_items	5,852
order_payments	5,472
products	1,500
sellers	312

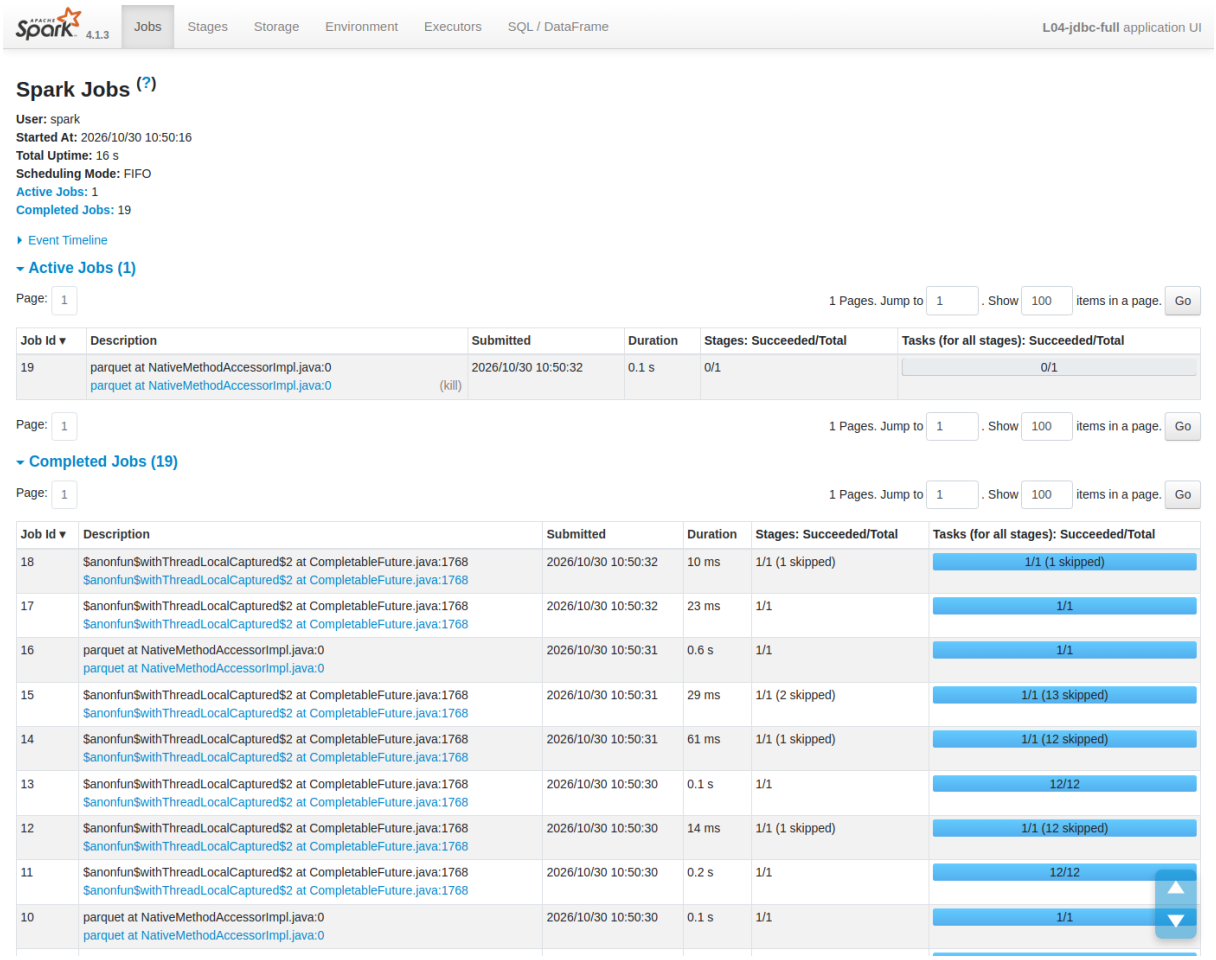


Hình 10. Kết quả `spark-submit --step full`: số partition, thời gian đếm, số dòng mỗi partition, kết quả ghi Parquet và physical plan có `PartitionFilters`.

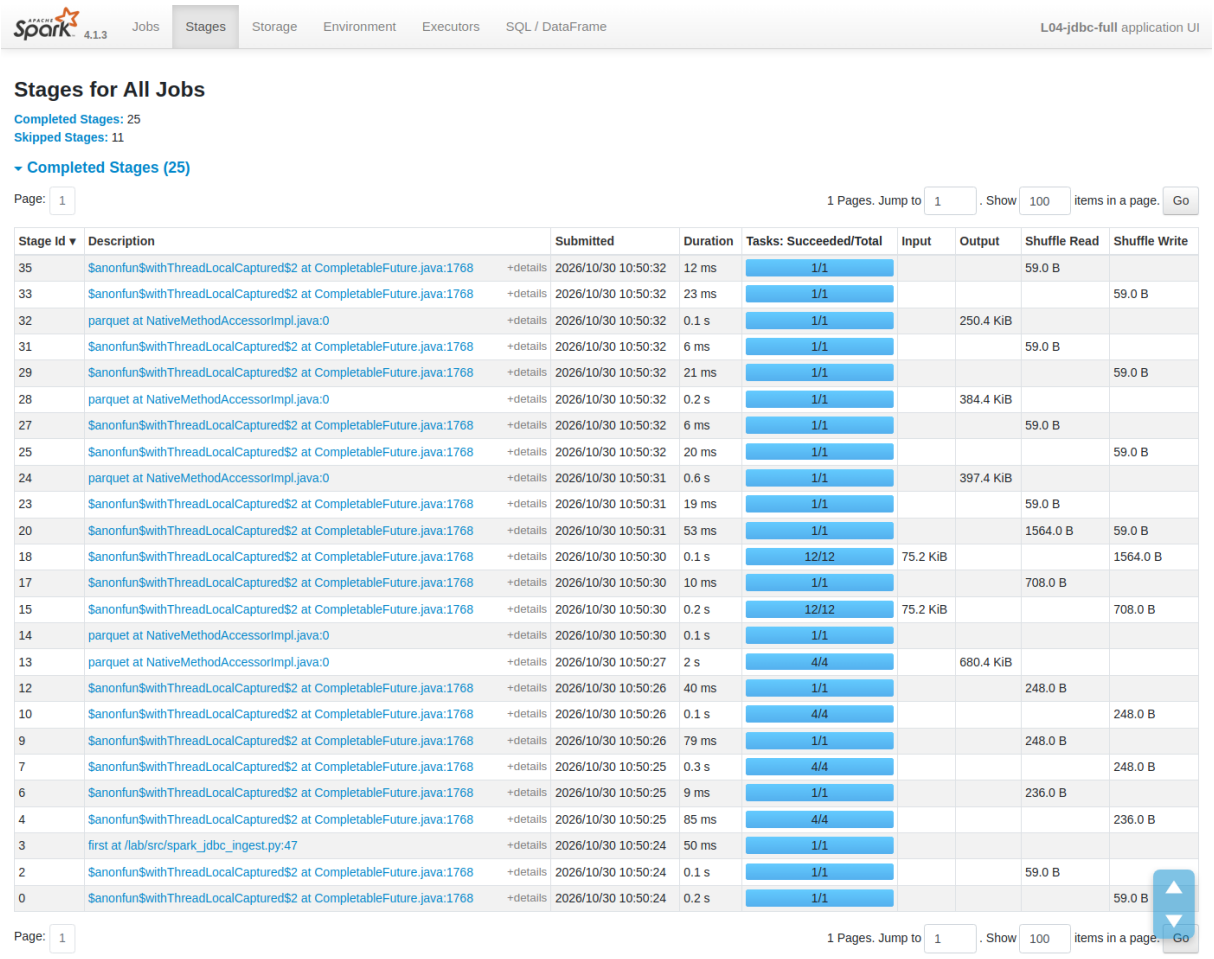


Hình 11. Nhật ký PostgreSQL ghi lại các câu `SELECT 1 FROM orders WHERE ...` theo khoảng do Spark sinh ra.

Trên Spark UI của ứng dụng `L04-jdbc-full`, các stage đọc song song có 4/4 task và stage ghi Parquet có 4/4 task với đầu ra 680.4 KiB (Hình 12, Hình 13).



Hình 12. Spark UI, trang Jobs của ứng dụng `L04-jdbc-full`.



Hình 13. Spark UI, trang Stages: các stage đọc JDBC song song và stage ghi Parquet đều có 4 task.

Kiểm tra bằng `hdfs dfs -ls` và `hdfs dfs -du` (Hình 14): `/data/olist/spark/orders` có 22 mục (`_SUCCESS` và 21 thư mục `purchase_month=`), dung lượng 680.4 K, còn `/data/olist/sqoop/orders_full` là 1.0 M.

```
L04 — Parquet partitionBy(purchase_month) trên HDFS

$ hdfs dfs -ls /data/olist/spark/orders | head
Found 22 items
-rw-r--r--   1 spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/_SUCCESS
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-01
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-02
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-03
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-04
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-05
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-06
...
$ hdfs dfs -du -s -h /data/olist/spark/orders /data/olist/sqoop/orders_full
680.4 K  680.4 K  /data/olist/spark/orders
1.0 M    1.0 M    /data/olist/sqoop/orders_full
```

Hình 14. Đầu ra Parquet phân vùng theo tháng trên HDFS và so sánh dung lượng với đầu ra văn bản của Sqoop.

4.5 So sánh Sqoop và Spark JDBC

Bảng 9 là bảng so sánh của Phần 3 trong đề bài, bổ sung thời gian đo được trong bài này.

Bảng 9. So sánh Sqoop và Spark JDBC khi nạp bảng `orders`.

Tiêu chí	Sqoop 1.4.7 (MapReduce)	Spark JDBC
Tham số song song	`-m 4` + `--split-by`	`numPartitions=4` + `partitionColumn`
Cách tính khoảng	Sqoop tự chạy `SELECT MIN, MAX` (BoundingValsQuery)	người dùng truyền `lowerBound`/`upperBound` (chỉ là stride)
Nạp gia tăng	Có sẵn: `--incremental append/lastmodified`, saved job lưu last value	Tự viết: truy vấn watermark + file trạng thái
Xử lý UPDATE	`--merge-key` (thêm một MR job)	`row_number()` khử trùng lặp, hoặc MERGE vào table format
Định dạng đầu ra	Văn bản CSV, 1.0 MB	Parquet Snappy phân vùng theo tháng, 680.4 KB
Biến đổi khi nạp	Không (chỉ SQL qua `--query`)	Đầy đủ DataFrame API (`withColumn`, `partitionBy`, …)
Thời gian nạp `orders`	25.2 s MR job (28.5 s cả lệnh)	3.10 s ghi Parquet (21.2 s cả `spark-submit` 6 bảng, gồm khởi động JVM)
Trạng thái dự án	Apache Attic (ngừng từ 2021)	Đang phát triển

Sqoop chậm chủ yếu vì mỗi lệnh phải sinh mã, biên dịch lớp record và dựng một MapReduce job riêng với container YARN. Spark dùng một SparkSession cho cả 6 bảng và ghi thẳng ra định dạng cột có nén, nên đầu ra nhỏ hơn khoảng một phần ba.

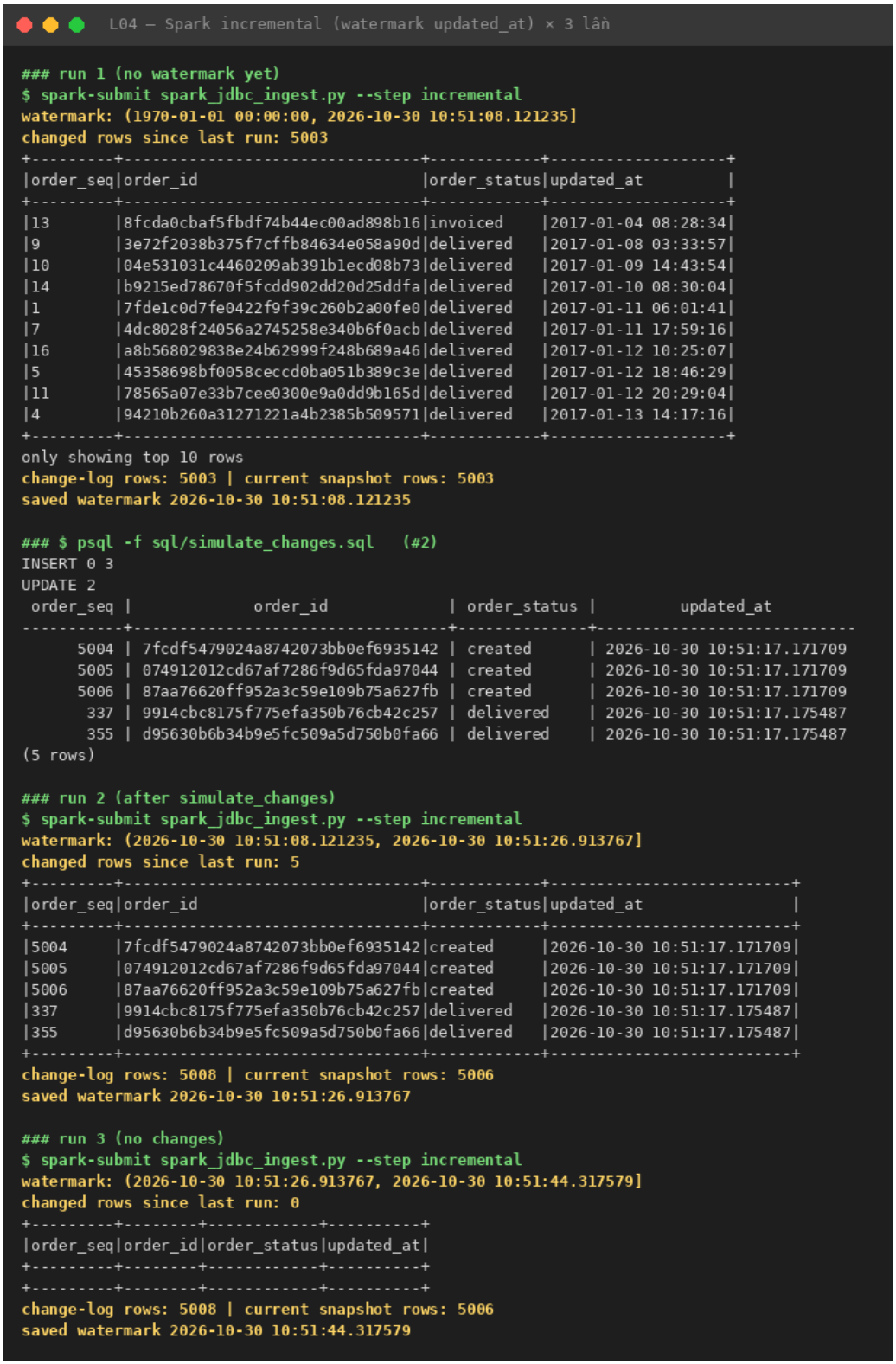
4.6 Spark nạp gia tăng bằng watermark

Bước `incremental` được chạy ba lần (Bảng 10, Hình 15).

Bảng 10. Kết quả ba lần chạy Spark nạp gia tăng (thời điểm ngày 2026-10-30).

Lần chạy	Cửa sổ watermark	Dòng đọc được	Change log	Snapshot
1 (chưa có watermark)	`(1970-01-01 00:00:00, 2026-10-30 10:51:08.121235]`	5,003	5,003	5,003
`simulate_changes` lần 2	tạo 5004–5006, cập nhật đơn 337 và 355			
2	`(10:51:08.121235, 10:51:26.913767]`	**5**	5,008	5,006
3 (không có thay đổi)	`(10:51:26.913767, 10:51:44.317579]`	0	5,008	5,006

Change log giữ mọi phiên bản: $5003 + 5 = 5008$ dòng. Snapshot dùng `row_number()` theo `order_id`, sắp `updated_at DESC, _ingested_at DESC`, nên chỉ còn phiên bản mới nhất của mỗi đơn: $5003 + 3 = 5006$, hai đơn được cập nhật không bị nhân đôi. Lần thứ ba đọc 0 dòng, tức watermark đã được lưu đúng và chạy lại không làm sai kết quả. Cận trên của lần trước trùng khít với cận dưới của lần sau.



Hình 15. Ba lần chạy Spark `--step incremental`: lần 1 đọc 5,003 dòng, lần 2 đọc 5 dòng thay đổi, lần 3 đọc 0 dòng.

5 Thảo luận

5.1 Sqoop chạy câu SQL nào trước khi import, và từ đó ra 4 split bằng cách nào?

Trước khi import, Sqoop chạy `SELECT MIN("order_seq"), MAX("order_seq") FROM "orders"` (Hình 3) và nhận (1, 5000). Bộ chia số nguyên cắt đoạn này thành 4 khoảng bằng nhau về giá trị, mỗi khoảng rộng khoảng $(5000 - 1)/4 \approx 1250$. Mỗi map chạy `SELECT ... WHERE order_seq >= lo AND order_seq < hi`, riêng khoảng cuối dùng <= để lấy cả giá trị MAX. Vì `order_seq` liên tục từ 1 đến 5000 nên mỗi map nhận đúng 1,250 dòng (Bảng 3). Điểm cần nhớ là Sqoop chia theo giá trị, không theo số dòng, và câu hỏi tiếp theo cho thấy hệ quả của việc đó.

5.2 Vì sao các file `customers` không đều, và nếu dùng `--split-by customer_id` thì sao?

Zip code của khách hàng Olist dồn về São Paulo (01xxx–19xxx), nên khoảng [1014, 24536) nhận 2,450 dòng trong khi [48057, 71578) chỉ có 427 dòng (Bảng 4, Bảng 5). Đây là data skew: task nặng nhất quyết định thời gian của cả job. Còn `customer_id` là chuỗi hash kiểu text. Mặc định Sqoop từ chối chia theo cột text và báo lỗi, trừ khi bật `-Dorg.apache.sqoop.splitter.allow_text_splitter=true`. Kể cả khi bật, việc chia theo thứ tự chuỗi phụ thuộc collation nên split vẫn có thể lệch hoặc rỗng. Cột chia tốt nhất là cột số, phân bố đều và có chỉ mục, như `order_seq`.

5.3 Với `lowerBound=1, upperBound=5003, numPartitions=4`, Spark có đọc dòng `order_seq` = 9,999 không?

Có đọc. `lowerBound` và `upperBound` chỉ dùng để tính bước chia, không phải bộ lọc. Nhật ký ở Hình 11 cho thấy partition cuối là `order_seq >= 3752` và không có cận trên, nên dòng 9,999 rơi vào partition cuối. Tương tự, giá trị nhỏ hơn `lowerBound` và NULL rơi vào partition đầu. Cận sai không làm mất dữ liệu, nhưng nếu nhiều dòng nằm ngoài cận thì partition hai đầu sẽ phình to và gây skew. Vì vậy script lấy MIN/MAX thật trước khi đọc, và nhờ đó 4 partition gần đều nhau (Bảng 7).

5.4 Vì sao phải chốt cận trên trước khi đọc, và chỉ lưu watermark sau khi ghi xong?

Script lấy `wm_new = now()` của cơ sở dữ liệu trước khi đọc, rồi truy vấn `updated_at > wm_old AND updated_at <= wm_new`. Nếu lấy `now()` sau khi đọc, dòng nào được cập nhật trong lúc đang đọc sẽ có `updated_at` nhỏ hơn watermark mới mà lại không nằm trong dữ liệu vừa đọc, và lần sau sẽ bỏ qua nó luôn. Cận trên cố định còn bảo đảm các partition JDBC song song

cùng nhìn một cửa sổ, và các cửa sổ liên tiếp liền nhau mà không chồng lên nhau. Bảng 10 cho thấy điều đó: lần 2 bắt đầu đúng tại 10:51:08.121235, lần 3 tại 10:51:26.913767.

Về thứ tự lưu, nếu job chết giữa chừng thì watermark cũ vẫn còn, lần chạy lại sẽ đọc lại cả cửa sổ. Dữ liệu có thể bị trùng nhưng không mất (at-least-once), và phần trùng được loại bằng `row_number()` theo `order_id` khi dựng snapshot. Ngược lại, nếu lưu watermark trước rồi mới ghi và bước ghi thất bại, cửa sổ đó sẽ không bao giờ được đọc lại.

5.5 Chế độ append hay `lastmodified` bắt được UPDATE, và chế độ nào bắt được DELETE?

`append` chỉ dựa vào `order_seq > last_value` nên chỉ bắt được INSERT: saved job lấy 3 dòng mới (Bảng 6). `lastmodified` dựa vào `updated_at` nên bắt được cả INSERT và UPDATE: 5 dòng gồm 3 đơn mới cùng đơn 139 và 278; khi đó cần `--merge-key` để gộp bản cũ với bản mới. Không chế độ nào bắt được DELETE, vì dòng đã xóa không còn trong bảng để truy vấn.

5.6 Watermark trên `updated_at` không giải quyết được gì, và CDC khác ở đâu?

Vấn đề đầu tiên là DELETE: dòng bị xóa không bao giờ xuất hiện trong truy vấn, nên bản sao trên HDFS giữ lại dữ liệu đã không còn ở nguồn, trừ khi ứng dụng dùng soft delete hoặc so sánh cả snapshot. Thứ hai là các lần UPDATE trung gian: nếu một đơn đổi từ `shipped` sang `delivered` rồi `canceled` trong cùng một chu kỳ, chúng tôi chỉ thấy trạng thái cuối. Ngoài hai điểm đó, bản thân `updated_at` có thể không đáng tin. Ứng dụng có thể quên cập nhật cột, đồng hồ máy chủ có thể lệch, hoặc một transaction dài commit sau khi watermark đã vượt qua timestamp của nó, và dòng đó bị sót. Cách này cũng tạo tải quét lên nguồn (cần chỉ mục trên `updated_at`), còn độ trễ thì bằng chu kỳ lập lịch.

CDC (Debezium, sẽ làm ở L08) không truy vấn bảng mà đọc WAL của PostgreSQL qua logical replication [7], [8]. Mọi INSERT, UPDATE và DELETE thành sự kiện theo thứ tự commit, kèm ảnh trước và sau, không phụ thuộc cột `updated_at`, gần như không tạo tải truy vấn và trễ chỉ vài giây.

5.7 Điều gì xảy ra với cơ sở dữ liệu nguồn khi đặt `-m 50` hoặc `numPartitions=50`?

Khi đó có 50 kết nối đồng thời, mỗi kết nối chạy một range scan. Con số này dễ chạm `max_connections` (PostgreSQL mặc định 100) và ứng dụng production sẽ không lấy được kết nối. Các truy vấn còn tranh CPU, I/O và buffer cache với tải OLTP; nếu cơ sở dữ liệu đã là nút thắt thì tốc độ gần như không tăng. Với bảng 5,000 dòng, mỗi task chỉ còn khoảng 100 dòng và chi phí khởi tạo lớn hơn lợi ích. Thực tế nên giữ khoảng 4–8 kết nối và thống nhất với DBA, đọc từ read replica hoặc ngoài giờ cao điểm, và đặt `fetchsize` hợp lý.

5.8 Hạn chế của thực nghiệm

Mọi thứ chạy trên một máy: HDFS một DataNode, YARN một NodeManager, Spark local mode, các task cùng dùng chung một máy và một cơ sở dữ liệu. Thời gian đo vì vậy phản ánh chi phí khởi động nhiều hơn thông lượng, và với khoảng 1 MB dữ liệu thì so sánh 25.2 s với 3.10 s chỉ nên hiểu theo nghĩa định tính. Thời gian `count()` chỉ đo một lần, và lần đọc song song chạy sau lần naive nên có thể được lợi từ JVM đã nóng; con số 0.22 s chưa đủ để kết luận về hiệu năng. Cuối cùng, do `simulate_changes.sql` chạy hai lần nên số dòng (5,003 trong Spark, 5,006 trong snapshot) khác với giá trị tham chiếu trong README.

6 Kết luận

Qua nhật ký SQL thật, bài này xác nhận cả Sqoop và Spark JDBC đều chia bảng theo khoảng giá trị của một cột. Sqoop tự tính cận bằng bounding query, còn Spark dùng cận do người dùng truyền vào và để hở partition hai đầu. Vì thế độ đều của các task phụ thuộc vào phân bố của cột chia: `order_seq` cho 4 phần gần bằng nhau, còn zip code (2,450 so với 427 dòng) và timestamp (782 so với 1,686 dòng) đều lệch. Ở phần nạp gia tăng, `append` chỉ bắt INSERT, còn `lastmodified` hay watermark trên `updated_at` bắt thêm UPDATE. Pipeline Spark chốt cận trên trước khi đọc, lưu watermark sau khi ghi và khử trùng lặp bằng `row_number()`; khi chạy lại nó cho kết quả đúng: 5 dòng rồi 0 dòng.

Nếu làm thật, chúng tôi sẽ dùng Spark JDBC thay Sqoop, chọn cột chia kiểu số có chỉ mục và lấy MIN/MAX thật làm cận, giới hạn số kết nối, đọc từ read replica và ghi ra Parquet phân vùng. Các bước tiếp theo có thể thử là dùng `predicates` để chia theo `order_status`, kiểm tra predicate pushdown, ghi vào table format có MERGE (Delta Lake, Iceberg), và chuyển sang CDC bằng Debezium (L08) để bắt DELETE cùng các phiên bản trung gian.

Tài liệu tham khảo

[1] The Apache Software Foundation, “Sqoop User Guide (v1.4.7),” 2017. https://sqoop.apache.org/docs/1.4

[2] The Apache Software Foundation, “JDBC To Other Databases,” Spark SQL Guide. https://spark.apache.org/docs/latest/sql-data-sources-jdbc.html

[3] M. Armbrust et al., “Spark SQL: Relational Data Processing in Spark,” in *Proc. ACM SIGMOD*, 2015, pp. 1383–1394.

[4] M. Zaharia et al., “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” in *Proc. USENIX NSDI*, 2012.

[5] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O’Reilly Media, 2017.

[6] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA: O’Reilly Media, 2015.

[7] The PostgreSQL Global Development Group, “Logical Decoding,” PostgreSQL Documentation. https://www.postgresql.org/docs/current/logicaldecoding.html

[8] Debezium Community, “Debezium Connector for PostgreSQL,” Debezium Documentation. https://debezium.io/documentation/reference/stable/connectors/postgresql.html

Phụ lục A: Danh mục tệp nộp kèm

Bảng 11. Danh mục tệp nộp kèm báo cáo L04.

Tệp	Nội dung
`sqoop_part1.sh`	Lệnh Sqoop phần 1: explore, nạp toàn phần, nạp `customers` theo zip code, `append`, `lastmodified` cơ sở, saved job
`spark_jdbc_ingest.py`	Đọc JDBC song song, ghi Parquet phân vùng, nạp gia tăng theo watermark, snapshot khử trùng lặp
`01_stack_and_source.txt`	`docker compose ps`, danh sách bảng, thống kê `orders`
`02_sqoop_part1.txt`	Log Sqoop phần 1, số dòng mỗi file, phân bố zip code, saved job
`03_postgres_log_sqoop_queries.txt`	Nhật ký PostgreSQL: bounding query và các câu SELECT của Sqoop
`04_simulate_changes_1.txt`	Kết quả `simulate_changes.sql` lần 1
`05_sqoop_part2_after_changes.txt`	Saved job, `lastmodified` + merge, export
`06_spark_full.txt`	Kết quả `spark-submit --step full`
`07_postgres_log_spark_queries.txt`	Nhật ký PostgreSQL: các truy vấn Spark (kèm một số truy vấn Sqoop append/saved job ở đầu)
`08_spark_output_hdfs.txt`	Liệt kê và dung lượng đầu ra Spark và Sqoop trên HDFS
`09_spark_incremental.txt`	Ba lần chạy `--step incremental` và `simulate_changes.sql` lần 2
`img/`	Ảnh chụp terminal, YARN UI, JobHistory và Spark UI

Phụ lục B: Trích đoạn mã nguồn

Các tệp được liệt kê ở Bảng 11. Đoạn đầu trong `sqoop_part1.sh` nạp `customers` chia theo zip code:

```
sqoop import "${CONN[@]}" --table customers --split-by
 ↪  customer_zip_code_prefix -m 4 "${NULLS[@]}" \
  --target-dir /data/olist/sqoop/customers_full --delete-target-dir
```

Trong `spark_jdbc_ingest.py`, bước `--step full` đọc song song theo `order_seq`, đọc theo timestamp rồi ghi Parquet phân vùng theo tháng:

```
orders = jdbc(spark, dbtable="orders", partitionColumn="order_seq",
              lowerBound=str(b.lo), upperBound=str(b.hi),
 ↪  numPartitions="4", fetchsize="1000")
by_ts = jdbc(spark, dbtable="orders",
 ↪  partitionColumn="order_purchase_timestamp",
             lowerBound="2017-01-01 00:00:00", upperBound="2018-09-01
 ↪  00:00:00",
             numPartitions="4")
out = orders.withColumn("purchase_month",
                        F.date_format("order_purchase_timestamp", "yyyy-MM"))
out.write.mode("overwrite").partitionBy("purchase_month").parquet(f"{BASE
 ↪  }/orders")
```

Bước `--step incremental` chốt cận trên bằng `now()` trước, sau đó mới đọc cửa sổ và nối vào change log:

```
wm_new = jdbc(spark, query="SELECT to_char(now(), 'YYYY-MM-DD
 ↪  HH24:MI:SS.US') AS ts").first().ts
changed_sql = (f"(SELECT * FROM orders WHERE updated_at > timestamp
 ↪  '{wm_old}'"
               f" AND updated_at <= timestamp '{wm_new}') AS
                ↪  changed_orders")
changes = jdbc(spark, dbtable=changed_sql) \
    .withColumn("_ingested_at", F.lit(wm_new).cast("timestamp"))
changes.write.mode("append").parquet(f"{BASE}/orders_changes")
```

Cuối bước này, script dựng snapshot từ change log, và watermark chỉ được ghi ở dòng cuối, sau khi snapshot đã ghi xong:

```
log = spark.read.parquet(f"{BASE}/orders_changes")
w = Window.partitionBy("order_id") \
    .orderBy(F.col("updated_at").desc(), F.col("_ingested_at").desc())
current = log.withColumn("_rn", F.row_number().over(w)).filter("_rn =
 ↪  1").drop("_rn")
current.write.mode("overwrite").parquet(f"{BASE}/orders_current")
spark.createDataFrame([(wm_new,)], "watermark string").coalesce(1) \
    .write.mode("overwrite").json(f"{BASE}/_state/orders_watermark")
```